

Projet Arma 3 — MARL de combat à rôles — DOSSIER COMPLET

Document de compilation regroupant **(1)** la transcription complète de la conversation de travail et **(2)** les trois documents de référence produits. Auto-suffisant : destiné à être archivé ou transmis.

- Sujet : former dans Arma 3 une équipe de 4 agents (1 chef + 3 spécialistes) capable d'opérer une mission de combat coopérative, par apprentissage multi-agents (MARL).
- Registre : niveau expert, théorie en profondeur, avec traduction systématique « en clair ».
- Date de compilation : 2026-06-01.

Note éditoriale : pour éviter la triple répétition, les développements théoriques les plus longs de la conversation sont reproduits sous forme condensée dans la transcription (Partie 1), et **intégralement** dans les annexes (Partie 2). Les passages uniques (description littéraire, recherche de projets existants, décisions) sont reproduits en entier dans la transcription.

PARTIE 1 — TRANSCRIPTION DE LA CONVERSATION

Échange 1 — Cadrage du sujet

Utilisateur : Je voudrais qu'on travaille ensemble les algorithmes comportementaux pour l'IA, à un niveau expert.

Claude : J'ai distingué trois familles très différentes derrière « algorithmes comportementaux » : (1) **architectures d'agents** (Behavior Trees, FSM/HFSM, GOAP, HTN, Utility AI, subsumption, BDI) ; (2) **apprentissage du comportement** (imitation : BC, DAgger, GAIL ; RL ; RL hiérarchique ; IRL) ; (3) **multi-agents / émergence** (flocking, stigmergie, théorie des jeux). J'ai posé trois questions de cadrage : domaine, approche (théorie vs implémentation), langage.

Réponses de l'utilisateur : - Domaine : machine Linux dédiée à l'IA, **couplée à Arma 3** ; objectif = former des agents capables d'opérer une **mission de combat avec des rôles distincts**. - Approche : **théorie en profondeur**. - Langage : **Python**.

Échange 2 — Structure de l'équipe et objectif

Utilisateur : Une équipe de 4 membres : 1 chef + 3 spécialisés, avec pour but d'atteindre l'objectif (qui peut être multiple). La réussite, c'est remplir la mission **et** rester en vie.

Claude (synthèse) : J'ai montré que cette phrase impose un **problème multi-critères conjonctif**, et détaillé les quatre façons de combiner « mission » et « survie » : - **somme pondérée** — piège : ne couvre que la partie convexe du front de Pareto ; - **contrainte / CMDP** — maximiser la mission sous contrainte de pertes $\leq$ seuil (recommandé) ; - **lexicographique** — priorité stricte ; - **sensible au risque (CVaR)** — optimiser la queue de distribution, pas l'espérance. J'ai introduit les **machines à récompense** pour « objectif multiple » et la **hiérarchie manager-worker** pour « 1 chef + 3 spécialistes ». Puis trois questions de modélisation.

Réponses de l'utilisateur : - Survie : **niveau équipe** (évite le « lazy agent »). - Combinaison mission/survie : **CMDP** (contrainte, lagrangien). - Rôles : **capacités fixes + tâches assignées dynamiquement par le chef**.

Échange 3 — Modèle consolidé + Module 1

Claude (synthèse) : Modèle verrouillé = **Dec-POMDP coopératif, sous contrainte, hiérarchique**, résolu en CTDE. Puis premier module en profondeur (reproduit intégralement en **Annexe A, Module 1**) : le dilemme CTDE, le **principe IGM**, la hiérarchie d'expressivité **VDN $\subset$ QMIX $\subset$ QPLEX**, la limite de monotonie (tactiques non-monotones), et l'extension au CMDP par **double critique + lagrangien**.

Échange 4 — Théorisation de l'ensemble des modules

Utilisateur : Je voudrais que tu théorises l'ensemble des modules.

Claude : Production du corpus complet (8 modules) — reproduit **intégralement en Annexe A**. Synthèse en une carte : chaque module répond à une question du problème (formalisation → décomposition de valeur → assignation de crédit → rôles → hiérarchie → observabilité → récompense → interface Arma), puis l'architecture intégrée (acteur-critique hiérarchique sous contrainte) et les frontières (limite de représentation, perte du chef, non-stationnarité multi-niveaux, sécurité à l'entraînement, débit d'échantillons, reward hacking).

Échange 5 — Description littéraire (registre romancé)

Utilisateur : Décris-moi en langage littéraire ce projet.

Claude :

Les quatre âmes de silicium

Il y a, quelque part dans une machine de Linux qui ne dort jamais, un monde qui ne sert qu'à mourir. Arma 3 en dessine les collines, les murs criblés, les lignes de crête où le vent semble souffler pour personne. C'est là que naissent nos quatre soldats — non pas de chair, mais de nombres ; non pas une fois, mais mille fois, dix mille fois, dans une aube qui recommence sans fin.

Ils sont quatre. Un chef, et trois autres, chacun porteur d'un don qui ne lui sera jamais retiré : l'un sait soigner, l'autre sait faire taire l'ennemi sous le poids du feu, le troisième a sa propre vocation. Le chef, lui, ne tire pas mieux que les siens — son art est ailleurs. Il voit l'ensemble là où chacun ne voit qu'un fragment, et il distribue le destin : *toi, là ; toi, couvre ; toi, avance*. Sa parole n'est pas un ordre crié mais une intention silencieuse qui descend la chaîne, et la troupe se met en mouvement comme un seul corps aux membres séparés.

Car telle est leur condition tragique et magnifique : chacun ne perçoit que sa parcelle de monde. Le brouillard de la guerre n'est pas une métaphore, ici — c'est une loi mathématique. Nul ne possède la carte entière. Chacun agit dans la pénombre de son propre regard, devine ce qu'il ne voit pas, se souvient de ce qu'il a vu, et fait confiance — à la radio, aux trois autres, à une cohérence qui n'existe nulle part et partout à la fois.

Ils apprennent par la mort. C'est leur seule pédagogie. Au commencement règne le chaos : ils s'égaillent, s'exposent, tombent absurdement, et le monde redémarre. Mais chaque chute laisse une trace infime, un poids déplacé dans la grande matrice de leur jugement. À force de périr, ils découvrent — sans que jamais personne le leur dise — qu'il vaut mieux bondir ensemble que s'élancer seul, qu'un flanc se déborde à deux, qu'un blessé se relève si l'on revient le chercher. La coordination n'est pas inscrite en eux : elle *émerge*, comme une eau qui finit par trouver son lit.

Et sur tout cela veille une double loi, presque biblique dans sa simplicité : **accomplir la mission, et rester en vie**. Deux commandements qui se contredisent à chaque pas, car la victoire appelle l'audace et la survie commande la prudence. Entre les deux se tient un arbitre invisible — un nombre, λ , le prix du risque — qui monte quand le sang coule et redescend quand la troupe se fait trop frileuse. Il est leur conscience comptable, la pesée perpétuelle du courage contre la perte.

Ce que nous faisons, au fond, n'a pas de nom neuf. Nous enseignons à des âmes de silicium le plus ancien des arts humains : se battre ensemble, obéir et commander, sacrifier sans gaspiller, ramener les siens. Nous ne leur donnons ni doctrine ni manuel — seulement un monde, une épreuve qui se répète, et la patience de les laisser devenir, à travers d'innombrables petites morts, quelque chose qui ressemble à une équipe.

Échange 6 — Traduction « en clair » des formules

Utilisateur : Traduis le modèle expert avec les formules en langage littéraire mais pas romancé, pour aborder les choses autrement (je ne suis pas mathématicien).

Claude : Production de la traduction en clair — reproduite **intégralement en Annexe B**. Principe : nommer chaque objet mathématique, puis dire en mots ce qu'il affirme exactement.

Échange 7 — Besoins techniques pour l'entraînement

Utilisateur : J'apprends beaucoup plus comme ça. Consigne-le dans le dossier. Et de quoi ce projet a-t-il besoin en termes de code, de bibliothèques, etc. pour l'entraîner ?

Claude : Sauvegarde de la version « en clair » + réponse technique — reproduite **intégralement en Annexe C**. Idée directrice : deux chantiers (le pont Arma sur-mesure ; le moteur d'apprentissage, où des bibliothèques mûres existent), et un conseil décisif : développer sur un **simulateur-jouet rapide** avant de brancher Arma.

Échange 8 — Consignation

Utilisateur : Oui, consigne ça dans des documents.

Claude : Création des trois fichiers de référence (corpus théorique, en clair, mise en place).

Échange 9 — Préparation de la session sur la machine Linux

Utilisateur : J'attends de rentrer chez moi pour te brancher à ma machine Linux et mon Claude qui tourne sur cette machine.

Claude : Avertissement clé : les documents et la mémoire sont **locaux à ce Mac** ; il faudra **copier les trois fichiers** sur la machine Linux. Rappel : le simulateur-jouet ne nécessite pas Arma, donc on peut démarrer le code sans le pont.

Échange 10 — Piloter Linux depuis le Mac

Utilisateur : Est-ce possible de faire le pont entre vous ? L'idée : piloter la machine Linux depuis ce Mac.

Claude : Pas de canal « Claude-à-Claude » natif. La vraie solution = **un seul Claude qui commande + un accès distant** : - **Option A (recommandée)** : Claude Code tourne **sur** Linux, piloté depuis le Mac par SSH/Remote-SSH ; accès natif complet aux fichiers, à Arma et au GPU. - **Option B** : ce Claude-ci (sur le Mac) commande Linux par SSH — bien pour des commandes rapides, moins ergonomique pour l'édition de fichiers distants. (*Question posée, puis mise en attente par l'utilisateur.*)

Échange 11 — Existe-t-il des projets de ce type ? (recherche web)

Utilisateur : Sais-tu me dire s'il existe des projets de ce type — Arma multi-agent ?

Claude (résultats de recherche) :

Réponse courte : un projet exactement comme celui-ci — agents MARL de combat, à rôles, entraînés *dans* Arma 3 — **n'existe pas publiquement en version prête à l'emploi**. Les grands écosystèmes MARL (PettingZoo, MALib, RLlib) intègrent StarCraft II, Google Research Football, OpenSpiel — pas Arma.

Ce qui existe avec Arma + apprentissage (usages différents) : - **RAND** — Arma 3 comme générateur d'**images synthétiques** pour entraîner de la reconnaissance de cibles (vision, pas agents). -

Arma3MLArt (GitHub) — un **calculateur de tir** (balistique) par ML ; mono-tâche. - **Forums Bohemia** — discussions communautaires (« ML/AI projects using Arma », « Neural network SQF example ») ; bricolage. - **Mods d'IA Arma** (VCOM, LAMBS, ASR AI) — **scriptés / à base de règles**, pas d'apprentissage.

Là où le projet existe vraiment (hors Arma) — défense / wargaming, souvent sur VBS : - « **Scaling RL through Feudal Multi-Agent Hierarchy** » (DTIC) — exactement la structure chef/manager (hiérarchie féodale). - « **Mastering the Digital Art of War** » (arXiv 2408.13333) — agents de combat par RL hiérarchique (décrit d'après titre/résumé, PDF non ouvert). - « **Coordinating Multi-Agent Deep RL in Wargame** » (ACM) — MADRL, entraînement centralisé / décision décentralisée (CTDE). - RAND, programme finlandais, Frazer-Nash (RU) — MARL appliqué aux simulateurs militaires.

Conclusion : la partie « algorithmes » est éprouvée (hiérarchie, MARL coopératif, CTDE) ; la partie « pont vers Arma 3 » est la vraie nouveauté et l'apport original du projet. Cela valide la stratégie retenue (machinerie d'abord sur simulateur-jouet, pont Arma ensuite).

Sources : - <https://github.com/ChosunOne/Arma3MLArt> - <https://forums.bohemia.net/forums/topic/221079-machine-learning-ai-projects-using-arma/> - <https://forums.bohemia.net/forums/topic/227270-neural-network-working-sqf-example/> - https://www.rand.org/content/dam/rand/pubs/research_reports/RRA600/RRA683-1/RAND_RRA683-1.pdf - <https://apps.dtic.mil/sti/trecms/pdf/AD1201723.pdf> - <https://arxiv.org/pdf/2408.13333> - <https://dl.acm.org/doi/fullHtml/10.1145/3446132.3446137> - <https://www.sciencedirect.com/science/article/pii/S1877050925031461> - <https://github.com/sjtu-marl/malib>

PARTIE 2 — DOCUMENTS DE RÉFÉRENCE (ANNEXES)

ANNEXE A — Corpus théorique complet (8 modules)

Cadre du projet. Former, dans Arma 3, une équipe de 4 agents (1 chef + 3 spécialistes à capacités fixes, tâches assignées dynamiquement) capable d'opérer une mission de combat coopérative. Objectif : **maximiser le succès de mission sous contrainte de pertes d'équipe** (CMDP). Mission compositionnelle. Modèle global : **Dec-POMDP coopératif, sous contrainte, hiérarchique**, résolu en CTDE.

Module 0 — Formalisation et complexité

Dec-POMDP $\langle I, S, \{A_i\}, T, R, \{\Omega_i\}, O, h, \gamma \rangle$; variante CMDP : $\max \mathbb{E}[\sum \gamma^t R]$ s.c. $\mathbb{E}[\sum \gamma^t C] \leq c$. Récompense d'équipe $\rightarrow$ Dec-POMDP (pas POSG). **Complexité : Dec-POMDP NEXP-complet** (Bernstein 2002) $\rightarrow$ on ne vise jamais l'optimum, on exploite la structure (rôles, hiérarchie). Deux difficultés : **non-stationnarité** et **assignation de crédit**. Paradigme **CTDE**.

Module 1 — Décomposition de valeur et principe IGM

IGM (Son 2019) : $\text{argmax}_a Q_{\text{tot}} = (\text{argmax } Q_1, \dots, \text{argmax } Q_n) \rightarrow$ l'argmax local de chacun reconstitue l'optimum joint. Hiérarchie : **VDN** (additif, Sunehag 2018) $\subseteq$ **QMIX** (monotone, $\partial Q_{\text{tot}}/\partial Q_i \geq 0$, hyperréseau conditionné sur s, Rashid 2018) $\subseteq$ **QPLEX** (classe IGM complète, Wang 2021). **Limite de QMIX** : ne représente pas les payoffs non-monotones (coordination tout-ou-rien — ex. débordement avec vs sans couverture) $\rightarrow$ tactiques de combat génériquement non-monotones. **CMDP** : deux critiques (retour Q^R , coût Q^C), couplage lagrangien $\mathcal{L} = Q^R - \lambda(Q^C - c)$, montée-descente duale, λ = prix du risque auto-réglé.

Module 2 — Policy gradient multi-agents et assignation de crédit

Gradient $\nabla J = \mathbb{E}[\nabla \log \pi_i \cdot A_i]$ avec **critique centralisé**. **MADDPG** (Lowe 2017, déterministe, continu). **COMA** (Foerster 2018) : **avantage contrefactuel** $A_i = Q(s,a) - \sum \pi_i Q(s, (a_{-i}, a_i))$ = récompense de différence (Wolpert & Tumer). **MAPPO** (Yu 2021) : backbone empirique recommandé. Variante contrainte : MAPPO-Lagrangian, MACPO (Gu 2021). Avantages PG pour ce projet : pas besoin d'IGM, gère l'hétérogénéité, s'intègre à la hiérarchie/lagrangien.

Module 3 — Théorie des rôles

Trois vues : rôle = **restriction d'action** (RODE, Wang 2021), **latent émergent** (ROMA, Wang 2020), **type/capacité fixe** (ce projet). Ici : capacité fixe r_i + tâche dynamique $g_i^t \rightarrow \pi(a_i|o_i, r_i, g_i^t)$, dynamlicité portée par le chef. **Partage de paramètres + embedding de rôle** (Gupta 2017) = compromis efficacité/spécialisation.

Module 4 — RL hiérarchique (le chef = manager)

Semi-MDP. **Options** $\langle I, \pi, \beta \rangle$ (Sutton, Precup, Singh 1999). **FeUdal Networks** (Vezhnevets 2017) ; **Feudal Multi-Agent Hierarchies** (Ahilan & Dayan 2019) = exactement la structure 1+3. **Option-Critic** (Bacon 2017) pour apprendre les options. Espace d'action du chef = « assigner g_i à i » (petit, structuré). CMDP $\times$ hiérarchie : le chef alloue un **budget de risque** entre sous-tâches. Problème ouvert : robustesse à la perte du chef.

Module 5 — Observabilité partielle, mémoire, communication

Croyance $b(s) = P(s|\text{historique})$. En Dec-POMDP : croyance **locale** par agent. **Mémoire** : réseau récurrent (DRQN, Hausknecht & Stone 2015 ; R2D2, Kapturowski 2019). **Communication apprise** : DIAL (Foerster 2016), CommNet (Sukhbaatar 2016), TarMAC (Das 2019) $\rightarrow$ la radio = canal appris à bande passante limitée. Fog of war = **valeur de la reconnaissance** (réduction d'entropie).

Module 6 — Récompense : éparses, shaping, CMDP, curriculum

Shaping par potentiel $F = \gamma\Phi(s') - \Phi(s)$ (Ng, Harada, Russell 1999) = seule forme prouvablement sûre. **Machines à récompense** (Toro Icarte 2018/2022) = outil pour mission compositionnelle (automate + QRM). **HER** (Andrychowicz 2017). **Curriculum** (PLR, Jiang 2021 ; PAIRED, Dennis 2020). **Motivation intrinsèque** (ICM, Pathak 2017 ; RND, Burda 2018). **Imitation/IRL** (GAIL, AIRL). **CVaR** (Chow 2015) pour l'aversion au risque.

Module 7 — L'interface Arma 3 comme décision de modélisation

Le **niveau d'abstraction d'action définit le MDP**. Sweet spot : **macro-actions = options à intra-politique = Behavior Tree doctrinal** (le RL apprend *quand*, le BT exécute *comment*). Échelle de temps (temps réel, accélération, headless, instances parallèles). Le pont (callExtension → socket). **Goulot = débit d'échantillons** (Arma lent).

Synthèse — architecture intégrée

Acteur-critique **hiérarchique, sous contrainte, en CTDE** : machine à récompense (étapes) → chef-manager (options, planificateur sous contrainte, budget de risque) → 3 spécialistes (politique partagée conditionnée par capacité fixe + tâche, encodeur récurrent, macro-actions en BT doctrinal, radio apprise) → deux critiques centralisés décomposés couplés par λ ; crédit par avantage contrefactuel. Entraînement : curriculum + motivation intrinsèque + amorçage par imitation ; instances Arma headless parallèles. Exécution : décentralisée, chacun sur sa croyance locale.

Frontières / problèmes ouverts

Représentation non-monotone ; perte d'agents (ad hoc teamwork, Stone 2010 ; Other-Play, Hu 2020) ; non-stationnarité multi-niveaux ; sécurité pendant l'entraînement ; complexité d'échantillon vs débit Arma ; reproductibilité ; généralisation inter-missions ; reward hacking.

Bibliographie pivot

Bernstein 2002 ; Oliehoek & Amato 2016 ; Sunehag 2018 ; Rashid 2018/2020 ; Son 2019 ; Wang 2021 (QPLEX) ; Lowe 2017 ; Foerster 2018 ; Yu 2021 ; Wolpert & Tumer 2002 ; Wang 2020 (ROMA) ; Wang 2021 (RODE) ; Gupta 2017 ; Sutton/Precup/Singh 1999 ; Bacon 2017 ; Vezhnevets 2017 ; Ahilan & Dayan 2019 ; Hausknecht & Stone 2015 ; Kapturowski 2019 ; Foerster 2016 ; Sukhbaatar 2016 ; Das 2019 ; Ng/Harada/Russell 1999 ; Toro Icarte 2018/2022 ; Andrychowicz 2017 ; Pathak 2017 ; Burda 2018 ; Ho & Ermon 2016 ; Altman 1999 ; Achiam 2017 ; Gu 2021 ; Chow 2015 ; Jiang 2021 ; Dennis 2020 ; Stone 2010 ; Hu 2020.

ANNEXE B — Le modèle expert, traduit en clair (sans mathématiques)

Ce que décrit la formule du problème. $\langle I, S, \{A_i\}, T, R, \{\Omega_i\}, O, h, \gamma \rangle$ = l'inventaire du jeu : les acteurs, l'état complet (caché) du monde, les actions possibles, la règle d'évolution (= Arma 3, qui la joue), la récompense, et ce que chacun perçoit. γ : une réussite immédiate vaut un peu plus qu'une réussite tardive. **E (espérance)** : « en moyenne, sur un très grand nombre de parties ».

Les deux lois. $\max \mathbb{E}[\sum \gamma^t R] \text{ s.c. } \mathbb{E}[\sum \gamma^t C] \leq c$ = « accomplir la mission le mieux possible, en moyenne, à condition que les pertes attendues ne dépassent pas un seuil que tu fixes ». La survie n'est pas un objectif de même rang : c'est une **limite à ne pas franchir**.

Pourquoi renoncer à la perfection. P / PSPACE / **NEXP** : trouver la stratégie d'équipe parfaite est d'une difficulté démesurée, démontrée inatteignable. On vise l'excellent atteignable, via la structure.

La valeur d'un geste (Q). Une note qui évalue « faire tel geste dans telle situation » par tout le bien que cela rapportera ensuite. **IGM** : si chacun choisit ce qui paraît le meilleur de son point de vue, le résultat doit coïncider avec le meilleur choix d'équipe. **Addition** (aveugle aux interactions) ; **monotonie** (« améliorer un seul membre ne peut que faire monter la note d'équipe ») ; **limite** : un homme qui s'élance seul, sans couverture, est pire que rien — ce que la monotonie ne sait pas représenter.

Apprendre et attribuer le mérite. Gradient de politique : « ajuster le comportement par petites touches, dans le sens qui en moyenne a fait mieux que prévu ». **Avantage contrefactuel (COMA)** : mesurer la contribution d'un soldat en comparant ce qui s'est passé à ce qui se serait passé, en moyenne, s'il avait agi autrement (les autres faisant la même chose).

Le curseur du risque (lagrangien). Noter un plan par sa valeur de mission moins une pénalité proportionnelle au dépassement du seuil de pertes. λ = prix du risque qui **se règle tout seul** : trop de pertes → il monte (prudence) ; pertes sous le seuil → il redescend (audace). Tu fixes une limite, pas un compromis figé.

Qui est qui, qui commande. $\pi(a|o_i, r_i, g_i)$ = le geste dépend de ce qu'on perçoit, de qui on est (capacité fixe), et de la tâche reçue. **Options** $\langle I, \pi, \beta \rangle$ = quand une manœuvre commence, comment elle se déroule, quand elle s'achève. Le chef déclenche des manœuvres qui durent, à cadence lente.

Agir sans tout voir. **Croyance** = estimation probable de l'entourage à partir de tout ce qu'on a vu et entendu → besoin de mémoire ; la reconnaissance a une valeur en soi.

Encourager sans corrompre le but. **Façonnage par potentiel** = seule manière prouvée de distribuer des encouragements en chemin sans détourner le but final. Garde-fou contre le « soldat zélé qui perd la guerre ».

ANNEXE C — Mise en place technique

Idée directrice. Deux chantiers : (1) le **pont vers Arma 3** (sur-mesure, difficile) ; (2) le **moteur d'apprentissage** (bibliothèques mûres). Recommandation majeure : développer d'abord sur un **simulateur-jouet rapide**, brancher Arma seulement à la fin.

Couche 1 — Simulateur (Arma 3), sur-mesure. Serveur dédié Linux *headless* ; mission SQF avec **reset rapide** ; pont via `callExtension` + extension native (`.so`) écrite avec `arma-rs` (Rust) ou **Intercept** (C++), dialoguant avec Python par **ZeroMQ**/sockets ; extraction d'observation en SQF (`knowsAbout` , `nearTargets` , santé, munitions) ; application d'actions (`doMove` , `doTarget` , `commandFire`) avec `disableAI` ; **accélération** (`setAccTime`) + instances parallèles.

Couche 2 — Interface standard. **PettingZoo** (multi-agents), **Gymnasium** (mono-agent).

Couche 3 — Moteur d'apprentissage. **EPyMARL** (prototype, colle à la théorie : QMIX/VDN/MAPPO/COMA/ MADDPG) → **Ray RLlib** / **MARLLib** (échelle, parallélisme) ; **CleanRL** (comprendre) ; **Stable-Baselines3** (premier prototype mono-agent).

Couche 4 — Contrainte (CMDP). **OmniSafe** ; **MAPPO-Lagrangian** / **MACPO**.

Couche 5 — Socle et outillage. **PyTorch** (JAX/JaxMARL non pertinent : Arma est un sim externe) ; **Hydra** ; **Weights & Biases** / **TensorBoard** ; **Docker**.

Matériel. Le frein est le **CPU** (instances Arma), pas le GPU. → beaucoup de cœurs + RAM généreuse + GPU correct. À vérifier : combien d'instances Arma de front.

Ordre de construction. Étape 0 : pont nu (SQF ↔ Python). Étape 1 : un agent, PPO (SB3). Étape 2 : simulateur-jouet 4 agents, MAPPO + conditionnement par rôle (EPyMARL). Étape 3 : contrainte (λ , OmniSafe). Étape 4 : chef + macro-actions/BT. Étape 5 : bascule vers Arma.

Fin du dossier.